HANDS-ON 02 | Procesamiento con RDD

GUÍA DEL ESTUDIANTE · CASO COMPLETAMENTE RESUELTO

Campo	Descripción
Curso	Base de Datos Avanzadas y Big Data
Unidad / semana	III. Big Data / Semana 12
Duración	4 horas (240 minutos)
Tecnología	Windows 11 · Anaconda · JupyterLab · PySpark
Saber esencial	Spark en clúster y operaciones con RDD

1. Logro y producto esperado

Al finalizar, el estudiante procesa registros mediante RDD, distingue transformaciones de acciones, experimenta con particiones y explica las limitaciones del modo local frente a un clúster con varias computadoras.

Producto: notebook ejecutado, comprobaciones de resultados, respuestas justificadas y reflexión sobre los tiempos de ejecución.

2. Situación problemática

Una organización recibe registros simulados de consumo de Agua, Energía y Residuos en cuatro regiones. Necesita obtener indicadores simples por sector y región. El equipo técnico desea comprender cómo Spark organiza las tareas y qué cambia cuando aumenta el volumen de información.

Todos los datos son simulados con fines didácticos. Las unidades de consumo son unidades convencionales; no se deben comparar como si representaran directamente metros cúbicos, kWh o toneladas.

3. Recursos y archivos

Archivo	Uso
HandsOn02_ESTUDIANTE.ipynb	Caso guiado y desafío de aplicación
datos_consumo_24.csv	Verificación manual: 24 filas
datos_consumo_100k.csv	Experimentos: 100 000 filas
generar_datos_hands_on02.py	Recrear los CSV con resultados idénticos

Requisito: seleccionar en JupyterLab el kernel del entorno Conda que contiene PySpark. La versión debe ser compatible con Python y Java instalados. Consultar la documentación oficial enlazada al final.

4. Activación de saberes previos (15 min)

Pregunte: ¿qué diferencia existe entre una tabla en memoria y una colección distribuible? ¿Por qué un cálculo puede requerir varias tareas? ¿Qué puede ocurrir si se lleva un archivo completo a la memoria de Python?

5. Ruta de trabajo (240 min)

Momento	Actividad	Min
Inicio	Problema y conceptos previos	25
Explorar	Verificar JupyterLab, crear sesión local y leer CSV	35
Implementar	Caso guiado: RDD, filter, map y reduceByKey	75
Experimentar	100 000 filas y particiones 1, 2, 4	45
Transferir	Desafío con región y suma por sector	40
Cierre	Evidencias y reflexión	20

6. Conceptos esenciales (lenguaje sencillo)

Concepto	Explicación
RDD	Colección de datos que Spark puede dividir en particiones para procesarlas.
Transformación	Operación que describe un nuevo RDD, como map o filter.
Acción	Operación que solicita un resultado, como count o take.
Partición	Porción de un RDD que puede procesarse mediante una tarea.
reduceByKey	Combina valores asociados a una misma clave.
Modo local	Ejecución en una computadora; no demuestra ejecución en varias máquinas.

7. Preparación en Anaconda y JupyterLab

Abra Anaconda Prompt, active el entorno ya utilizado en ETL y compruebe la instalación antes de ejecutar el cuaderno. Sustituya nombre_entorno por el nombre real.

```
conda activate nombre_entorno
python --version
java -version
python -m pip show pyspark
jupyter lab
```

Si PySpark no está instalado, revise las versiones compatibles y utilice el administrador de paquetes elegido para el entorno. No combine instalaciones de pip y conda sin revisar dependencias. En JupyterLab seleccione el kernel del entorno correspondiente.

8. Caso completamente resuelto: 24 registros

PASO 1. Crear una sesión de Spark. ¿Para qué? Para disponer del contexto que administra los RDD. El notebook ejecuta Spark en modo local con dos núcleos lógicos de trabajo (`local[2]`).

```
from pyspark.sql import SparkSession
spark = (SparkSession.builder.master("local[2]")
         .appName("UPN-S12-RDD").getOrCreate())
sc = spark.sparkContext
```

PASO 2. Leer el CSV con la función guiada del notebook. La lectura se realiza con el módulo csv de Python para que el estudiante identifique cada registro. Esta decisión es solo didáctica: primero los datos se cargan al proceso Python; no es una lectura distribuida de archivos.

```
registros = leer_registros("datos_consumo_24.csv")
print(len(registros))   # Resultado esperado: 24
```

PASO 3. Crear el RDD y verificar las particiones.

```
rdd = sc.parallelize(registros, 3)
print(rdd.getNumPartitions())  # 3
print(rdd.count())             # 24
```

PASO 4. Filtrar el sector Agua. filter describe el filtro; count solicita el resultado.

```
agua = rdd.filter(lambda x: x[3] == "Agua")
print(agua.count())            # 8
```

PASO 5. Contar y sumar por sector. La clave es el sector y el valor es una pareja (consumo, 1). reduceByKey combina ambos elementos.

```
parejas = rdd.map(lambda x: (x[3], (x[4], 1)))
resumen = parejas.reduceByKey(
    lambda a, b: (a[0] + b[0], a[1] + b[1]))
print(sorted(resumen.collect()))
```

Sector	Filas	Suma de consumo
Agua	8	164
Energia	8	172
Residuos	8	180

Interpretación: todos los sectores presentan ocho registros en el ejemplo pequeño. Se admite collect porque el resultado agregado contiene únicamente tres pares; no debe generalizarse a la extracción de todo un conjunto grande.

9. Escalar el caso: 100 000 registros

El notebook repite el conteo por sector sobre 100 000 registros. Se mantiene la misma idea de transformación y agregación; lo que cambia es el número de registros procesados.

Sector	Filas esperadas
Agua	33334
Energia	33333
Residuos	33333
TOTAL	100000

Compruebe resultados con `assert`. La verificación del número de filas no sustituye una justificación técnica de cómo se realizó la operación.

10. Experimento con particiones

Ejecute el mismo filtro usando 1, 2 y 4 particiones; repita cada medición tres veces. Observe los tiempos sin anticipar cuál será menor. Los resultados dependen del equipo, carga de trabajo y sobrecoste de ejecución.

Abra la interfaz Spark UI (normalmente http://localhost:4040 mientras la sesión está activa), si se encuentra disponible, y observe jobs, stages y tareas. Su presentación y puerto pueden variar.

11. Desafío autónomo (sin solución en material estudiante)

A. Determine el número de registros por región usando map y reduceByKey. B. Obtenga la suma de consumo por sector. C. Explique cuándo usar collect y cuándo evitarlo. D. Interprete su experimento con particiones con evidencias.

12. Evidencias y criterios de valoración

Criterio	Evidencia observable
Ejecución	Notebook ejecutado con resultados coherentes.
Comprensión	Diferencia correctamente transformaciones y acciones.
Validación	Comprueba cantidades y totales con resultados esperados.
Argumentación	Justifica particiones, limitaciones de collect y modo local.
Comunicación	Explica resultados de manera clara; declara ayuda de IA si la utilizó.

13. Preguntas de reflexión y cierre

¿Por qué una transformación no produce necesariamente un cálculo inmediato? ¿Por qué no es prudente ejecutar collect sobre todas las filas? ¿Qué demuestra el modo local y qué no? ¿Qué evidencia necesitaríamos para afirmar que una configuración es más rápida?

Referencias técnicas verificables

Apache Spark. RDD Programming Guide. https://spark.apache.org/docs/latest/rdd-programming-guide

Apache Spark. Installation — PySpark. https://spark.apache.org/docs/latest/api/python/getting_started/install.html

Observación: los requisitos de Java y Python varían según la versión de PySpark; revisar la documentación de la versión realmente instalada.